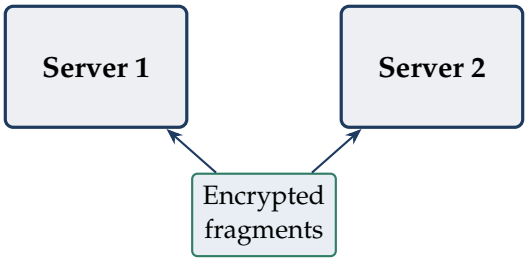
A Final-Year Undergraduate Project

A Secure Erasure Code–Based Cloud Storage System with Secure Data Forwarding

Asif Alif[1], Paresh Chandra Barman[1*]

[1]*Department of Information and Communication Technology, Islamic University, Kushtia 7003, Bangladesh*

A system that integrates primary encryption, erasure coding, threshold proxy re-encryption, and secure data forwarding to deliver confidential, robust, distributed cloud storage.



B.Sc. in Information and Communication Technology, Islamic University, Kushtia 7003, Bangladesh

Abstract

Cloud storage is a service model in which data is maintained, managed, and backed up remotely and made available to users over a network. Storing data off-site makes it accessible from anywhere without the burden of maintaining local storage and file-serving infrastructure; however, entrusting data to a third-party cloud raises serious concerns about confidentiality. Conventional encryption schemes protect confidentiality but restrict the functionality of the storage system, because only a limited set of operations can be performed over encrypted data.

The objective of this project is to construct a secure storage system that supports multiple functions simultaneously. We propose a re-encryption scheme and integrate it with an erasure code so that a secure distributed storage system is formulated. The system supports secure and robust data storage and retrieval. Its main technical contribution is that the re-encryption scheme supports encoding operations over encrypted messages as well as forwarding operations over encoded and encrypted messages. The method therefore integrates encryption, encoding, and forwarding into a single pipeline: the message is first encrypted, the erasure code then splits it into multiple fragments, those fragments are re-encrypted, and the re-encrypted fragments are distributed across independent storage servers. The erasure code guarantees data robustness, while the layered encryption and distribution guarantee confidentiality even when a single server is compromised.

Keywords: Cloud storage; Erasure coding; Re-encryption; Encryption; Decryption; Secure data forwarding; Distributed storage.

Table of Contents

Abstract **i**

1 Introduction **1**
1.1 Background . . . 1
1.2 Motivation . . . 1
1.3 Problem Statement . . . 2
1.4 Aims and Objectives . . . 2
1.5 Scope and Limitations . . . 3

2 Literature Review **4**
2.1 Cloud Storage Systems . . . 4
2.2 Erasure-Coding Techniques . . . 4
2.3 Secure Data-Forwarding Mechanisms . . . 4

3 The Existing System **6**
3.1 Overview of the Existing System . . . 6
3.2 Disadvantages of the Existing System . . . 6

4 Proposed System Methodology **7**
4.1 Overview of the Proposed System . . . 7
4.2 System Architecture . . . 7
4.3 Data Flow Diagram . . . 8
4.4 Advantages of the Proposed System . . . 9
4.5 Components and Modules . . . 9
4.5.1 Cloud Data Storage Module . . . 9
4.5.2 Data Encryption Module . . . 10
4.5.3 Erasure-Coding Module . . . 10
4.5.4 Re-encryption Module . . . 10
4.5.5 Data Forwarding and Retrieval Module . . . 10

5 Requirements **11**
5.1 Hardware Requirements . . . 11
5.2 Software Requirements . . . 11

6 Implementation **12**
6.1 Automation Script . . . 12
6.2 Primary Encryption . . . 14
6.3 Erasure Coding: Splitting the File . . . 16

6.4 Re-encryption 17
6.5 File Transfer 19
6.6 File Retrieval 21
6.7 Decryption 22
6.8 Merging the Fragments 24
6.9 Final Decryption 25

7 Conclusion and Future Work **27**
7.1 Conclusion 27
7.2 Future Work 27

References **28**

CHAPTER 1

Introduction

Cloud storage systems have gained considerable popularity because of their scalability, cost-effectiveness, and convenience. Users of these systems can access their data at any time and from any location by storing it on remote servers. This same reliance on remote infrastructure, however, raises persistent concerns about data security and privacy. This chapter introduces the background, motivation, problem statement, aims, and scope of the project.

1.1 Background

In recent years, cloud storage systems have become a mainstream way to store and share data because of their scalability, cost-effectiveness, and convenience. By placing their data on distributed servers, users can reach it at any time and from anywhere. The growing dependence on cloud storage has, in turn, intensified concerns about data security and privacy. Vulnerabilities such as unauthorised access, data loss, and data leakage undermine traditional approaches to storing data in the cloud.

To address these concerns, the combination of erasure coding and re-encryption is a promising solution for preserving the confidentiality, integrity, and availability (the CIA triad) of data. Erasure coding splits data into multiple fragments, and a re-encryption technique encrypts those fragments before they are distributed across multiple servers.

1.2 Motivation

Erasure coding provides fault tolerance and robustness, but it also introduces challenges related to secure data forwarding. Data forwarding is the process of sending data from the client to the storage nodes in a typical cloud storage system. Without sufficient security measures, this process can be exposed to intrusions such as eavesdropping, data manipulation, and man-in-the-middle attacks.

The goal of this project is to design and build a secure erasure code–based cloud storage system that not only ensures data availability and durability but also provides secure data transfer. By mitigating the security risks associated with data transmission, the project aims to offer a trustworthy and dependable cloud storage option. The proposed system uses cryptographic primitives, secure communication, and an efficient erasure-coding scheme to meet these goals, and it re-encrypts the fragmented data before transmission to provide an additional layer of security. In short, the combination

of erasure coding, re-encryption, and secure data forwarding is what motivates this project: together they enhance the security and reliability of cloud storage systems.

1.3 Problem Statement

The widespread adoption of cloud storage brings clear advantages in scalability, cost-effectiveness, and accessibility; yet these systems frequently face serious security and privacy issues. Traditional cloud storage is exposed to data loss and unauthorised access, leaving sensitive user data vulnerable to breaches.

One of the central difficulties in cloud storage is maintaining data availability and durability in the presence of hardware failures or server outages. By distributing data across several storage nodes and allowing the system to reconstruct lost or corrupted fragments, erasure codes have emerged as a powerful solution to this problem. When it comes to secure data transfer, however, erasure codes introduce challenges of their own.

Specifically, erasure code–based cloud storage systems lack secure data-forwarding mechanisms. The process of transferring data from the client to the storage nodes is vulnerable to eavesdropping, data manipulation, and man-in-the-middle attacks when appropriate safeguards are not in place. The project therefore aims to develop a secure erasure code–based cloud storage system with a flexible and efficient data-forwarding method that preserves confidentiality and integrity during transmission, defends against unauthorised access and manipulation, and overcomes the limitations of existing systems.

1.4 Aims and Objectives

The aim of this project is to strengthen the privacy and security of cloud storage using erasure-coding techniques and re-encryption: to identify the vulnerabilities of existing systems and overcome them, to split data into several parts, and to re-encrypt those parts so that they can be stored on different servers with an additional layer of protection.

The objectives of the project are:

- to analyse the existing system and identify its vulnerabilities;
- to design a secure erasure code–based cloud system that ensures the confidentiality, integrity, and authenticity of stored data;
- to develop a secure data-forwarding mechanism that protects data stored on the servers;
- to compare the proposed system with the existing system and show how the vulnerabilities are overcome; and

- to provide guidance for setting up and operating secure data-forwarding and erasure code–based cloud storage systems.

1.5 Scope and Limitations

The limitations of this project are:

- The project is primarily concerned with the security of data forwarding during transmission. It does not address complementary issues such as fine-grained access control, user authentication, or data privacy beyond the transmission stage.
- The project assumes a reliable cloud storage environment and does not model the difficulties introduced by unreliable or actively hostile cloud providers.
- The system is implemented and evaluated in a controlled environment, so some real-world network conditions and performance fluctuations are not taken into account.

CHAPTER 2

Literature Review

This chapter reviews the three bodies of work on which the project builds: cloud storage systems, erasure-coding techniques, and secure data-forwarding mechanisms.

2.1 Cloud Storage Systems

Cloud storage systems allow users to store and access their data remotely through a network of distributed servers, rather than keeping it on a local computer. The concept has a long lineage: J. C. R. Licklider (1960) introduced the early vision of a networked system in which users could store and access data from different locations around the world [1], and later work such as Luby's Tornado codes (1998) made efficient data repair and retrieval in distributed storage practical [2]. Subsequent systems, including Tahoe [3], Farsite [4], OceanStore [5, 6], and POTSHARDS/Total Recall [7], explored how to build durable, available storage over untrusted or failure-prone infrastructure.

2.2 Erasure-Coding Techniques

Erasure coding (EC) is a method of data protection in which data is broken into fragments, expanded with redundant pieces, and stored across a set of different locations or media. The theoretical foundations were laid by Gallager (1960), whose work significantly influenced the adoption of coding as a reliable and efficient method for data protection and recovery, and by Reed and Solomon (1962), who developed the error- and erasure-correcting codes that bear their names. In the context of distributed storage, decentralised erasure codes have been shown to provide robustness with controllable redundancy [8, 9], and systems such as Plutus [10] demonstrated secure file sharing over untrusted storage.

2.3 Secure Data-Forwarding Mechanisms

Secure data-forwarding mechanisms are essential to protect against eavesdropping, data tampering, and man-in-the-middle attacks during transmission. Diffie and Hellman (1976) introduced public-key exchange, and Rivest, Shamir, and Adleman (1977) proposed the RSA algorithm, together establishing the cryptographic basis for secure forwarding. Of particular relevance to this project, proxy re-encryption [11, 12] allows a ciphertext encrypted under one key to be transformed for another recipient without exposing the plaintext, and Lin and Tzeng [13] combined a threshold proxy re-encryption

scheme with a decentralised erasure code to build a secure distributed storage system — the design most closely related to the one proposed here.

CHAPTER 3

The Existing System

3.1 Overview of the Existing System

In the existing system, encryption and encoding are combined in a straightforward way. General encryption schemes protect data confidentiality but also limit the functionality of the storage system, because only a few operations can be performed over encrypted data. Storing data in a third party's cloud without applying erasure coding raises serious concerns about confidentiality. Moreover, constructing a secure storage system that supports multiple functions is challenging when the storage is distributed and has no central authority.

3.2 Disadvantages of the Existing System

The straightforward integration of encryption and encoding has three principal drawbacks:

- The user has to perform most of the computation, and the communication traffic between the user and the storage servers is high.
- Storing data in a third party's cloud without erasure coding creates a serious confidentiality risk.
- Because no erasure coding or re-encryption is applied, a third-party cloud provider can potentially read the entire data set.

CHAPTER 4

Proposed System Methodology

4.1 Overview of the Proposed System

The proposed system introduces a re-encryption scheme and integrates it with a decentralised erasure code so that a secure distributed storage system is formulated. Using a threshold proxy re-encryption scheme, the system provides both secure data storage and secure data forwarding.

In this system a user can store data far more securely than in a typical cloud storage system. The data is first encrypted at the primary level; erasure coding then splits the encrypted data into several parts; and finally a re-encryption step is applied to those parts to make them more secure. Whereas a typical system stores the data on a single cloud server, here the re-encrypted fragments are distributed across different storage servers, so that compromising any single server does not reveal the original data.

The main technical contribution is that the proxy re-encryption scheme supports encoding operations over encrypted messages as well as forwarding operations over encoded and encrypted messages. This tight integration of encoding, encryption, and forwarding enables the storage system to meet the requirements of data robustness, data confidentiality, and data forwarding at once. Storage servers independently perform encoding and re-encryption, and key servers independently perform partial decryption; considering the system in this general, distributed setting allows a flexible trade-off between the number of storage servers and the degree of robustness.

4.2 System Architecture

The proposed system architecture is shown in Figure 4.1. A user wishes to store a message m_1. First-level encryption is applied with a key to obtain the encrypted message em_1. Erasure coding then splits em_1 into four fragments S_1, S_2, S_3, S_4. A re-encryption scheme re-encrypts these fragments with a key to produce Es_1, Es_2, Es_3, Es_4, which are distributed across two independent cloud storage servers: Es_2 and Es_4 are stored on Server 1, while Es_1 and Es_3 are stored on Server 2.

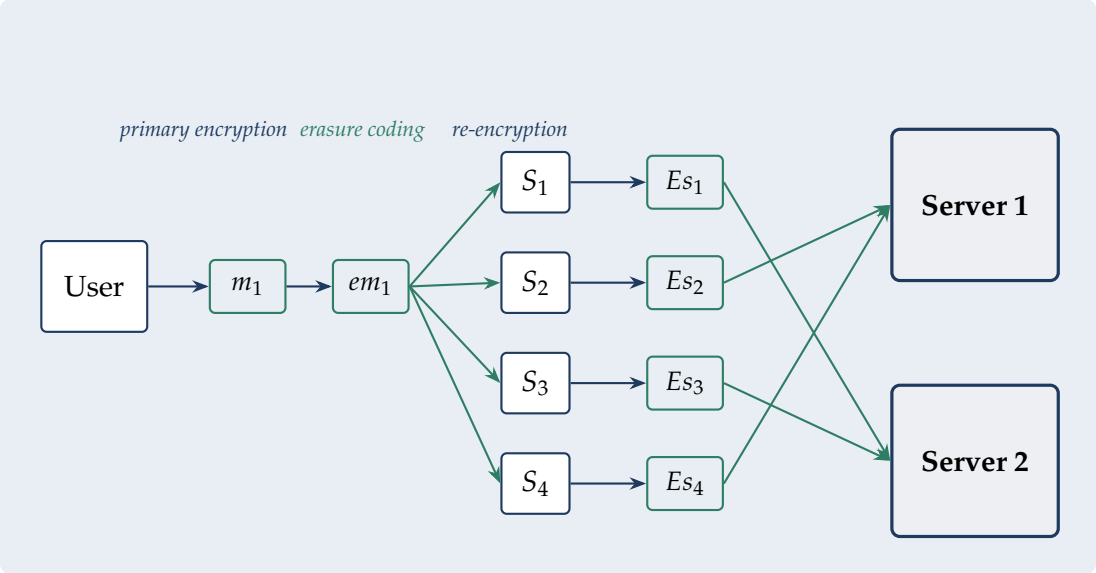


Figure 4.1: Proposed system architecture: encryption, erasure coding, re-encryption, and distribution across two independent storage servers.

When the user later wishes to retrieve the data, the system performs the reverse sequence: Es_1, Es_2, Es_3, Es_4 are decrypted with the key to recover S_1, S_2, S_3, S_4; the fragments are merged to reconstruct em_1; and primary-level decryption with the key yields the original data m_1.

4.3 Data Flow Diagram

The overall control and data flow of the system is shown in Figure 4.2. After the administrator logs in and the remote servers are configured, the user authenticates to the cloud and may upload, forward, or download files; each path records the appropriate storage details.

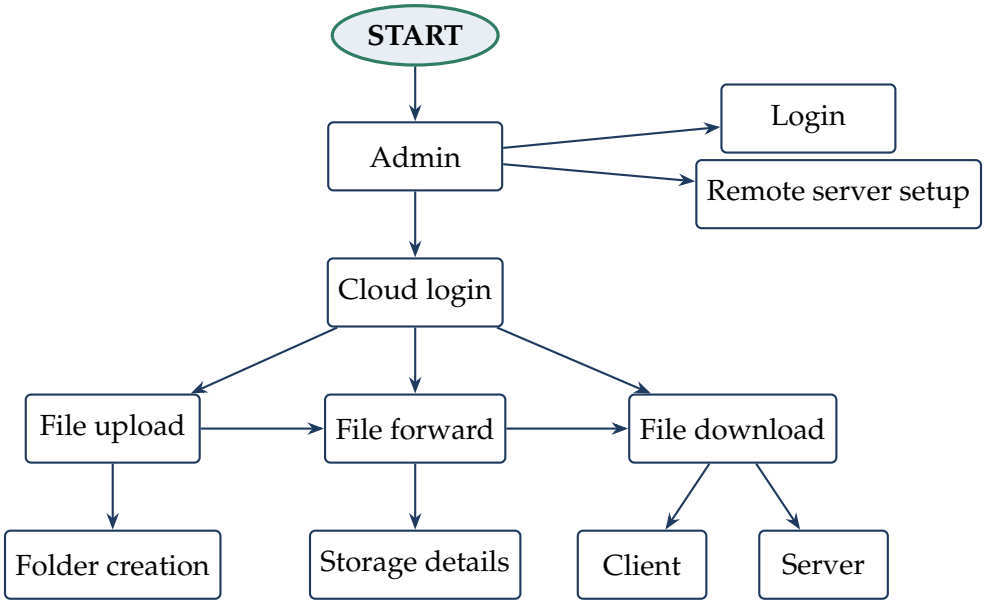


Figure 4.2: Data flow diagram of the proposed system.

4.4 Advantages of the Proposed System

- Primary-level encryption is applied to the data before any other processing.
- Erasure coding splits the data into parts that can be stored on different servers.
- The erasure code provides fault tolerance and additional security.
- A re-encryption scheme re-encrypts the fragments for an extra layer of protection.
- The tight integration of encoding, encryption, and forwarding lets the system meet the requirements of data robustness, confidentiality, and forwarding simultaneously.
- The design allows a flexible trade-off between the number of storage servers and the degree of robustness.

4.5 Components and Modules

This section describes the components and modules used to construct the proposed system.

4.5.1 Cloud Data Storage Module

The system uses two cloud storage servers. Together, the storage servers: form the distributed storage system in the cloud; store the data fragments generated by erasure coding; implement data redundancy and fault-tolerance mechanisms; and participate in data reconstruction and recovery.

4.5.2 Data Encryption Module

In this module the data is encrypted at the primary stage with a key. An integer key is used to encrypt the data at the primary level.

4.5.3 Erasure-Coding Module

The erasure-coding module: performs encoding and decoding operations to ensure fault tolerance and data recovery; distributes data fragments across multiple storage nodes; and implements efficient reconstruction algorithms to recover lost or corrupted fragments.

4.5.4 Re-encryption Module

The re-encryption module: re-encrypts the split data fragments; uses a key to re-encrypt those fragments; stores that key for future decryption; and thereby provides an additional layer of security.

4.5.5 Data Forwarding and Retrieval Module

The data forwarding and retrieval module: ensures the confidentiality, integrity, and authenticity of data during transmission; establishes secure channels between the client and the storage nodes; protects against a range of cyber attacks; and can securely forward data to others and retrieve it in its original form.

CHAPTER 5

Requirements

Two kinds of requirements must be met to implement the proposed system: hardware requirements and software requirements.

5.1 Hardware Requirements

The minimum hardware requirements are listed in Table 5.1.

Table 5.1: Minimum hardware requirements.

Component	Specification
Device	Laptop or desktop
RAM	8 GB
Hard disk	256 GB
Keyboard	Standard keyboard
Processor	Intel Core i5

5.2 Software Requirements

The software requirements are listed in Table 5.2.

Table 5.2: Software requirements.

Component	Specification
Operating system	Windows 10/11 (host)
Virtualisation	VMware or VirtualBox
User machine	Kali Linux
Storage servers	Ubuntu (or any Linux server)
Scripting	Python 3 and Bash

CHAPTER 6

Implementation

The complete workflow is automated by a Bash script that orchestrates a set of Python programs. The user machine is a Kali Linux virtual machine, and two Ubuntu virtual machines act as the storage servers. This chapter presents the automation script and then walks through each stage of the pipeline: primary encryption, erasure-coding (splitting), re-encryption, file transfer, retrieval, decryption, merging, and final decryption.

> *Note on credentials. For clarity the implementation originally hard-coded the storage-server credentials in the transfer scripts. In the listings below these passwords are redacted; in a production deployment they should be supplied through SSH key authentication or a secrets manager rather than embedded in source code.*

6.1 Automation Script

The entire project is driven by a single Bash script, `script7.sh`, which presents an eight-option menu and calls the relevant Python program for each stage. The script is shown in Listing 6.1.

Listing 6.1: The automation script `script7.sh`.

```
#!/bin/bash

# Primary encryption / final decryption
execute_primary_encryption_decryption() {
  python /root/Desktop/Finalyear/hello.py
}

# Erasure coding: split the file into fragments
execute_split_file() {
  read -p "Enter the file to split: " file_to_split
  read -p "Enter the size per file (e.g., 1K, 1M, 1G): " size_per_file
  split -b "$size_per_file" "$file_to_split" "output_prefix"
  echo "File split completed."
}

# Re-encryption of the fragments
execute_re_encryption() {
  python /root/Desktop/Finalyear/encryptioncode.py
}

# Forward fragments to the storage servers
```

```
execute_file_transfer() {
  python /root/Desktop/Finalyear/trantwoserver.py
}

# Retrieve fragments from the storage servers
execute_receive_files() {
  python /root/Desktop/Finalyear/rectwoserver.py
}

# Final decryption of the retrieved fragments
execute_final_decryption() {
  python /root/Desktop/Finalyear/dec1.py
}

# Merge the decrypted fragments
execute_merge() {
  read -p "Enter the path and name of the merged file: " merged_file_path
  while true; do
    read -p "Enter the path of the split file to merge (or 'done' to finish): "
    split_file_path
    if [ "$split_file_path" = "done" ]; then
      break
    fi
    cat "$split_file_path"* >> "$merged_file_path"
  done
  echo "Files merged successfully."
}

# Main menu loop
while true; do
  echo "Select an option:"
  echo "1. Primary Encryption and Final Decryption"
  echo "2. Split the File"
  echo "3. Re-encryption"
  echo "4. File Transfer to the Servers"
  echo "5. Receive Files from the Servers"
  echo "6. Decryption"
  echo "7. Merge"
  echo "8. Exit"
  read -p "Enter your choice: " choice

  case $choice in
    1) execute_primary_encryption_decryption ;;
    2) execute_split_file ;;
    3) execute_re_encryption ;;
    4) execute_file_transfer ;;
    5) execute_receive_files ;;
    6) execute_final_decryption ;;
    7) execute_merge ;;
    8) echo "Exiting ..."; exit 0 ;;
    *) echo "Invalid choice. Please try again." ;;
  esac
  echo
  echo "Press Enter to continue ..."
```

```
  read
done
```

6.2 Primary Encryption

The first stage encrypts the input file at the primary level. Running `./script7.sh` on the Kali machine presents the eight-option menu (Figure 6.1). Selecting option 1 and then the encrypt action prompts for an integer key (here 23) and a filename (here `final.pdf`). The encryption routine performs a byte-wise XOR of the file with the key and writes the result to `CC-final.pdf` (Figure 6.2).

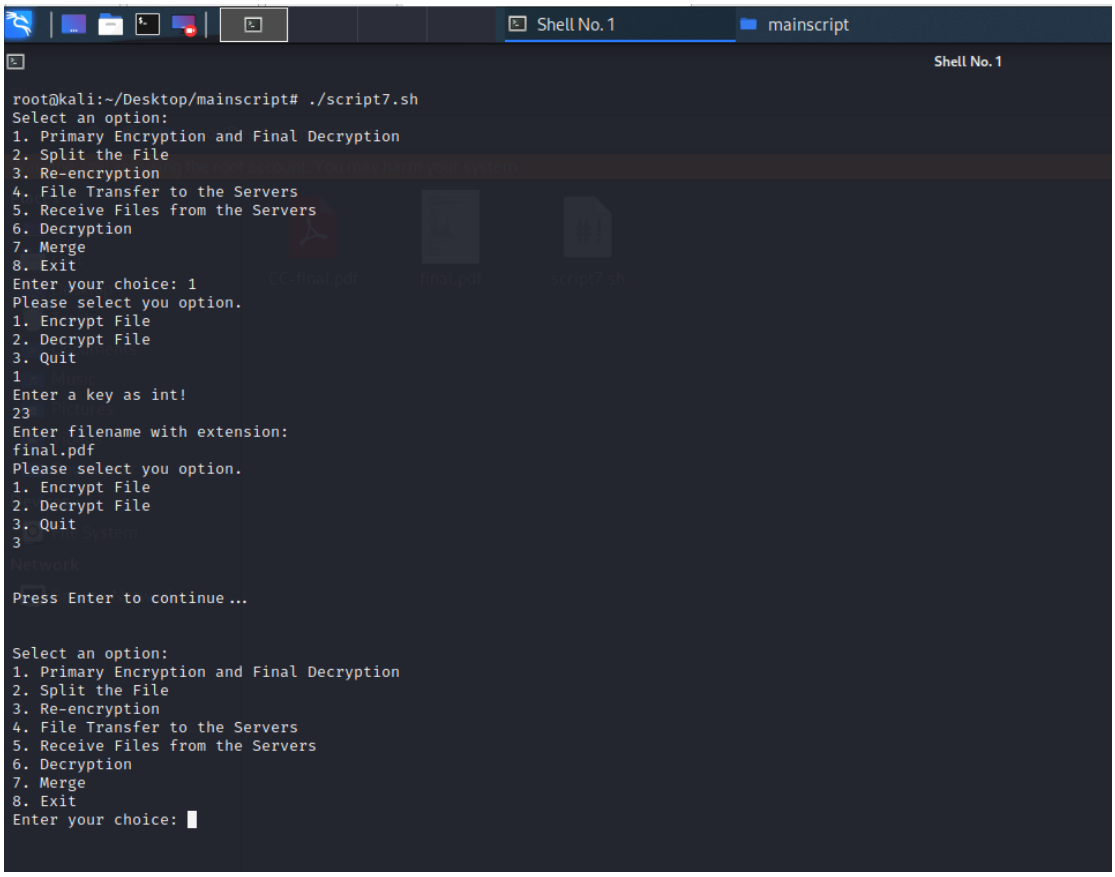


Figure 6.1: Running the automation script and selecting primary encryption.

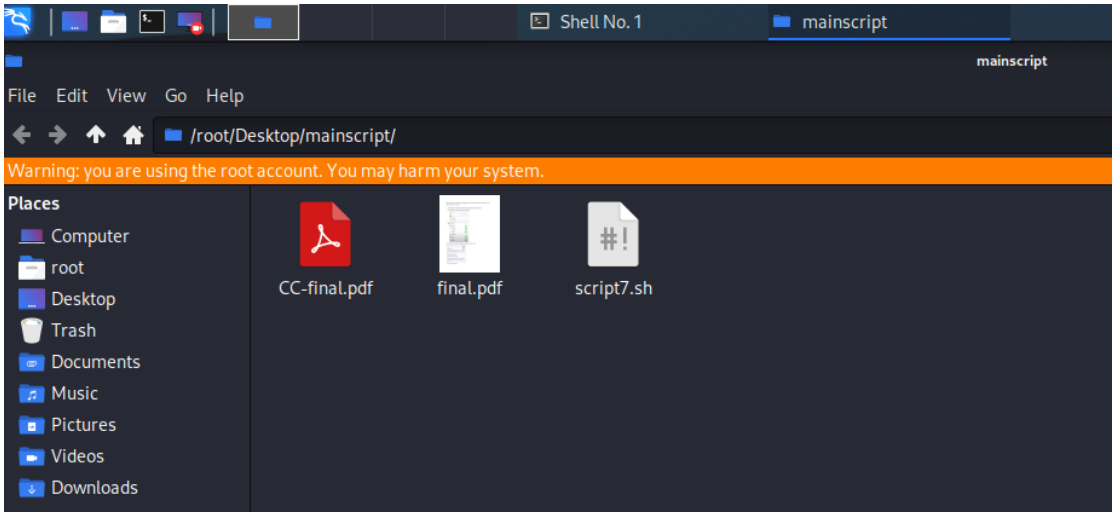


Figure 6.2: The encrypted output file `CC-final.pdf`.

The primary-encryption program, `hello.py`, is shown in Listing 6.2.

Listing 6.2: Primary XOR encryption/decryption (`hello.py`).

```
def Encrypt(filename, key):
  file = open(filename, "rb")
  data = file.read()
  file.close()

  data = bytearray(data)
  for index, value in enumerate(data):
    data[index] = value ^ key

  file = open("CC-" + filename, "wb")
  file.write(data)
  file.close()

def Decrypt(filename, key):
  file = open(filename, "rb")
  data = file.read()
  file.close()

  data = bytearray(data)
  for index, value in enumerate(data):
    data[index] = value ^ key

  file = open(filename, "wb")
  file.write(data)
  file.close()

choice = ""
while choice != "3":
  print("Please select an option.")
  print("1. Encrypt File")
  print("2. Decrypt File")
  print("3. Quit")
```

```
  choice = input()
  if choice == "1" or choice == "2":
    key = int(input("Enter a key as int!\n"))
    filename = input("Enter filename with extension:\n")
  if choice == "1":
    Encrypt(filename, key)
  if choice == "2":
    Decrypt(filename, key)
```

6.3 Erasure Coding: Splitting the File

The second stage applies erasure coding by splitting the encrypted file `CC-final.pdf` into fixed-size fragments. Selecting option 2 prompts for the file and the fragment size; for a 2.6 MB file a size of `1M` produces three fragments (Figure 6.3), named `output_prefixaa`, `output_prefixab`, and `output_prefixac` (Figure 6.4). This stage is implemented with the Linux `split` utility, integrated into the automation script.

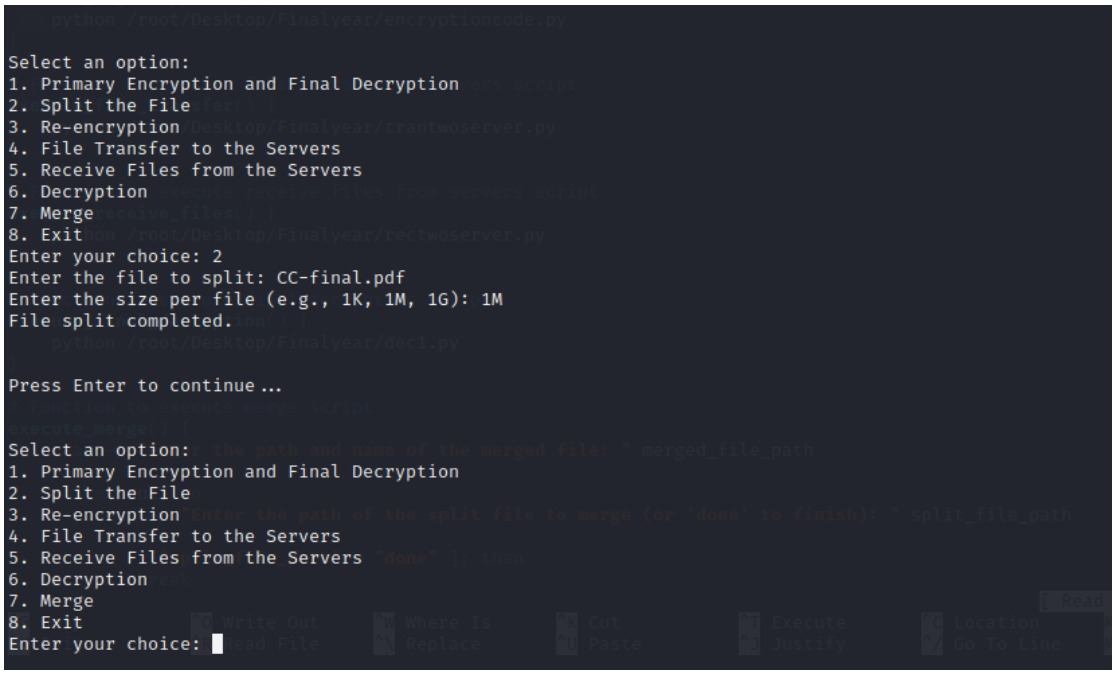


Figure 6.3: Splitting the encrypted file into fragments.

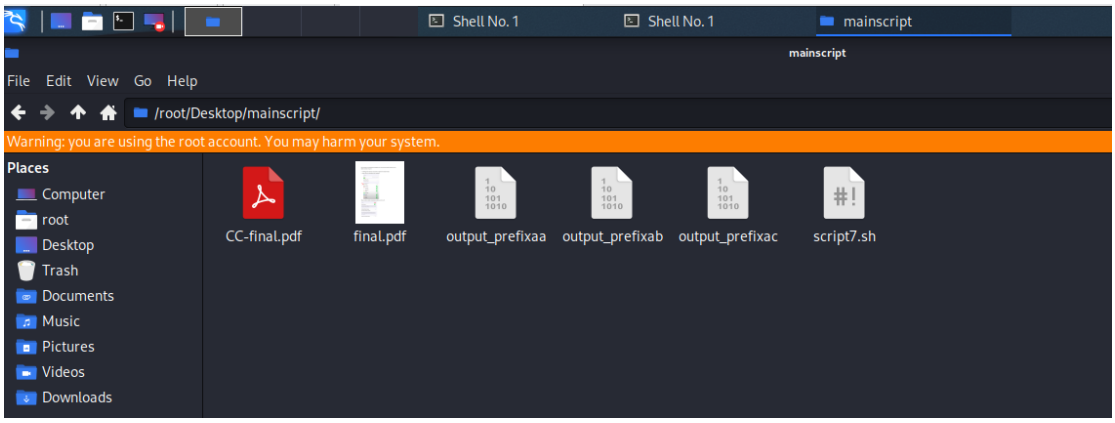


Figure 6.4: The three resulting fragments.

6.4 Re-encryption

The third stage re-encrypts the fragments to provide an extra layer of security. Selecting option 3 prompts for the number of fragments, their paths, and an encryption key; the program derives a Fernet key from the supplied pass-phrase, encrypts each fragment (producing `.enc` files), and saves the key to `encryption_key.xlsx` for later decryption (Figures 6.5 and 6.6).

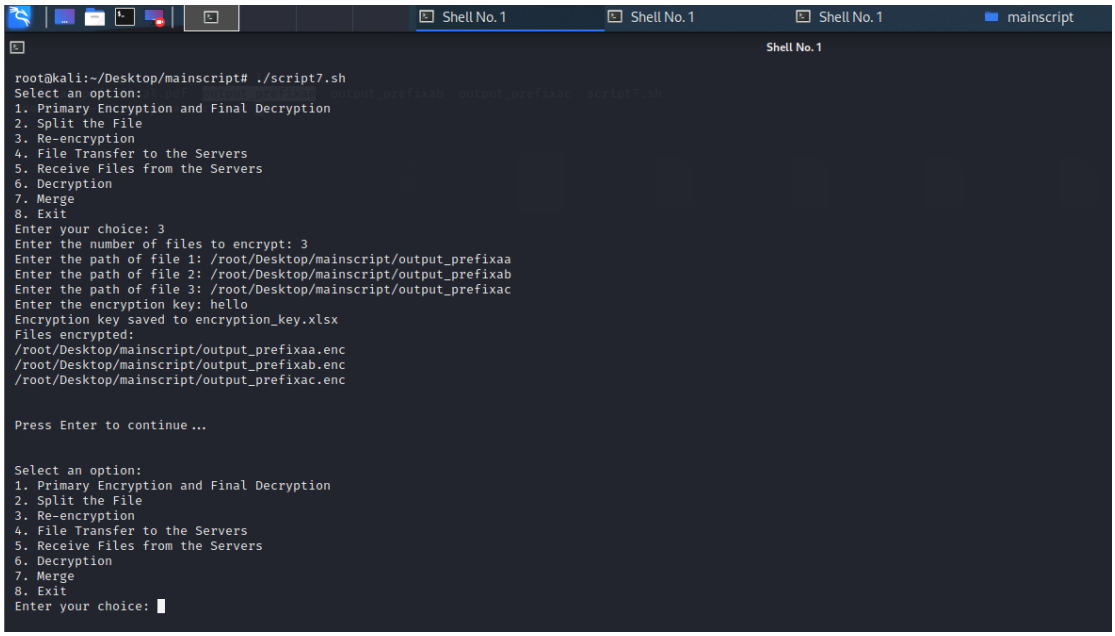


Figure 6.5: Re-encrypting the three fragments.

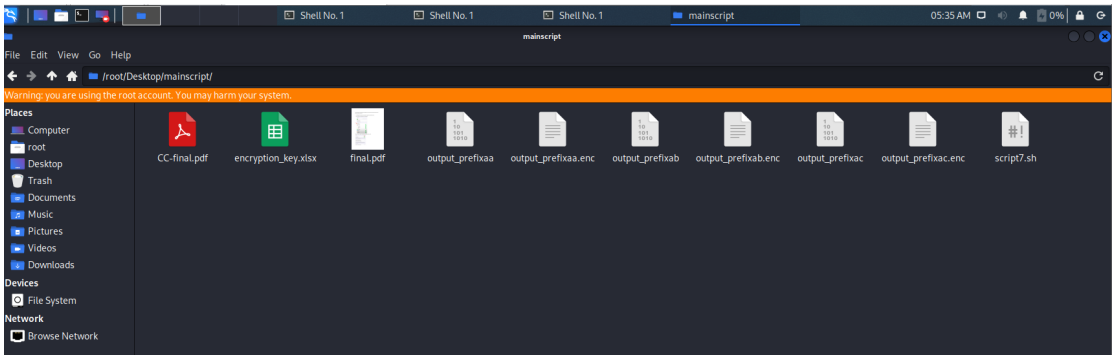


Figure 6.6: The re-encrypted fragments and the saved key file.

The re-encryption program, `encryptioncode.py`, is shown in Listing 6.3.

Listing 6.3: Re-encryption of the fragments (`encryptioncode.py`).

```
import os
import hashlib
import base64
import openpyxl
from openpyxl import Workbook
from cryptography.fernet import Fernet

def encrypt_file(filename, key):
  output_filename = filename + '.enc'
  with open(filename, 'rb') as file:
    data = file.read()
  cipher_suite = Fernet(key)
  encrypted_data = cipher_suite.encrypt(data)
  with open(output_filename, 'wb') as output_file:
    output_file.write(encrypted_data)
  return output_filename

def save_key_to_excel(key):
  workbook = Workbook()
  sheet = workbook.active
  sheet.title = 'Encryption Key'
  sheet['A1'] = 'Key'
  sheet['B1'] = key.decode()
  excel_filename = 'encryption_key.xlsx'
  workbook.save(excel_filename)
  return excel_filename

# Collect the fragment paths
filepaths = []
num_files = int(input("Enter the number of files to encrypt: "))
for i in range(num_files):
  filepath = input(f"Enter the path of file {i+1}: ")
  filepaths.append(filepath)

# Derive the encryption key from a pass-phrase
password = input("Enter the encryption key: ").encode()
key = base64.urlsafe_b64encode(hashlib.sha256(password).digest())
```

```
38
39  # Encrypt each fragment
40  encrypted_files = []
41  for filepath in filepaths:
42    encrypted_files.append(encrypt_file(filepath, key))
43
44  # Persist the key for later decryption
45  excel_filename = save_key_to_excel(key)
46  print(f"Encryption key saved to {excel_filename}")
47
48  print("Files encrypted:")
49  for encrypted_file in encrypted_files:
50    print(encrypted_file)
```

6.5 File Transfer

The fourth stage forwards the re-encrypted fragments to the two storage servers over SFTP. Selecting option 4 prompts for the number of files and their paths for each server; two fragments are sent to Server 1 and one to Server 2 (Figure 6.7). The fragments are confirmed as stored on Server 1 and Server 2 in Figures 6.8 and 6.9.

```
Enter the number of files to transfer to Server 1: 2
Enter the path of file 1 for Server 1: /root/Desktop/mainscript/output_prefixaa.enc
Enter the path of file 2 for Server 1: /root/Desktop/mainscript/output_prefixab.enc
Transferred: /root/Desktop/mainscript/output_prefixaa.enc ⟶ /home/crypticsec/Final_year_project/output_prefixaa.enc
Transferred: /root/Desktop/mainscript/output_prefixab.enc ⟶ /home/crypticsec/Final_year_project/output_prefixab.enc
Enter the number of files to transfer to Server 2: 1
Enter the path of file 1 for Server 2: /root/Desktop/mainscript/output_prefixac.enc
Transferred: /root/Desktop/mainscript/output_prefixac.enc ⟶ /home/crypticsec007/Final_year_project2/output_prefixac.enc
```

Figure 6.7: Forwarding the re-encrypted fragments to the servers.

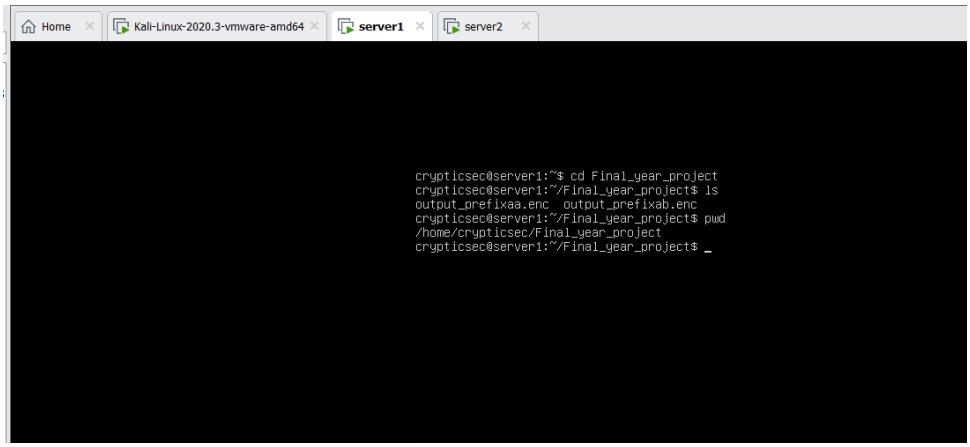


Figure 6.8: Fragments stored on Server 1.

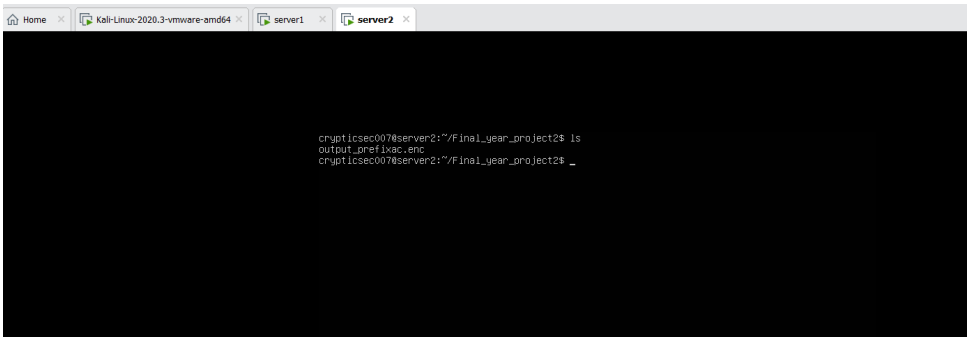


Figure 6.9: Fragment stored on Server 2.

The transfer program, `trantwoserver.py`, is shown in Listing 6.4; the server passwords are redacted.

Listing 6.4: Secure file transfer to the servers (`trantwoserver.py`).

```
import os
import paramiko

def transfer_files(source_files, destination_dir, hostname, username, password):
  # Connect to the server
  client = paramiko.SSHClient()
  client.load_system_host_keys()
  client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
  client.connect(hostname, username=username, password=password)

  # Create the destination directory if it does not exist
  client.exec_command(f'mkdir -p {destination_dir}')

  # Transfer each specified file over SFTP
  for source_file in source_files:
    filename = os.path.basename(source_file)
    remote_path = os.path.join(destination_dir, filename)
    sftp = client.open_sftp()
    sftp.put(source_file, remote_path)
    sftp.close()
    print(f"Transferred: {source_file} --> {remote_path}")

  client.close()

# ----- Server 1 -----
num_files_server1 = int(input("Enter the number of files to transfer to Server 1:
    "))
source_files_server1 = []
for i in range(num_files_server1):
  source_files_server1.append(input(f"Enter the path of file {i+1} for Server 1:
    "))
destination_directory_server1 = '/home/crypticsec/Final_year_project'
hostname_server1 = '192.168.152.129'
username_server1 = 'crypticsec'
password_server1 = '********'  # redacted
transfer_files(source_files_server1, destination_directory_server1,
```

```
        hostname_server1, username_server1, password_server1)

# ----- Server 2 -----
num_files_server2 = int(input("Enter the number of files to transfer to Server 2: "))
source_files_server2 = []
for i in range(num_files_server2):
  source_files_server2.append(input(f"Enter the path of file {i+1} for Server 2: "))
destination_directory_server2 = '/home/crypticsec007/Final_year_project2'
hostname_server2 = '192.168.152.130'
username_server2 = 'crypticsec007'
password_server2 = '********'  # redacted
transfer_files(source_files_server2, destination_directory_server2,
        hostname_server2, username_server2, password_server2)
```

6.6 File Retrieval

The fifth stage retrieves the fragments from the servers. Selecting option 5 prompts for the number of files and their remote paths; the two fragments on Server 1 and the one on Server 2 are downloaded to `/root/Desktop/Received/` (Figures 6.10 and 6.11).

```
Enter the number of files to transfer from Server 1: 2
Enter the remote path of file 1 from Server 1: /home/crypticsec/Final_year_project/output_prefixaa.enc
Enter the remote path of file 2 from Server 1: /home/crypticsec/Final_year_project/output_prefixab.enc
Transferred: /home/crypticsec/Final_year_project/output_prefixaa.enc ⟶ /root/Desktop/Received/output_prefixaa.enc
Transferred: /home/crypticsec/Final_year_project/output_prefixab.enc ⟶ /root/Desktop/Received/output_prefixab.enc
Enter the number of files to transfer from Server 2: 1
Enter the remote path of file 1 from Server 2: /home/crypticsec007/Final_year_project2/output_prefixac.enc
Transferred: /home/crypticsec007/Final_year_project2/output_prefixac.enc ⟶ /root/Desktop/Received/output_prefixac.enc
```

Figure 6.10: Retrieving the fragments from the servers.

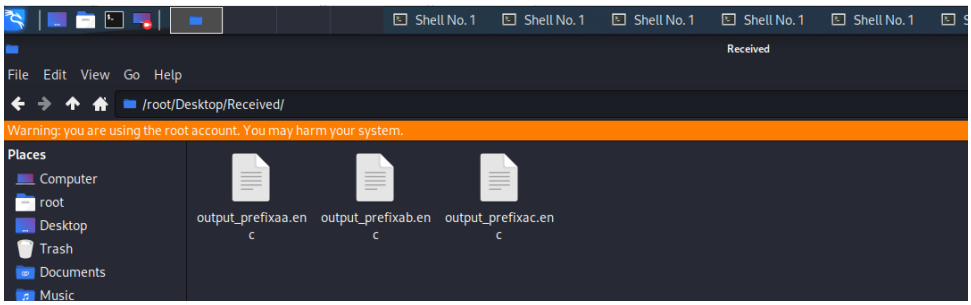


Figure 6.11: The retrieved fragments stored locally.

The retrieval program, `rectwoserver.py`, is shown in Listing 6.5; the server passwords are redacted.

Listing 6.5: Secure file retrieval from the servers (`rectwoserver.py`).

```
import os
import paramiko

def transfer_files(source_files, destination_dir, hostname, username, password):
```

```
    # Create the local destination directory if it does not exist
    os.makedirs(destination_dir, exist_ok=True)

    for source_file in source_files:
        _, filename = os.path.split(source_file)
        remote_path = source_file
        local_path = os.path.join(destination_dir, filename)

        # Connect to the server and download over SFTP
        client = paramiko.SSHClient()
        client.load_system_host_keys()
        client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
        client.connect(hostname, username=username, password=password)

        sftp = client.open_sftp()
        sftp.get(remote_path, local_path)
        sftp.close()
        print(f"Transferred: {source_file} --> {local_path}")
        client.close()

# ----- Server 1 -----
num_files_server1 = int(input("Enter the number of files to transfer from Server 1: "))
source_files_server1 = []
for i in range(num_files_server1):
    source_files_server1.append(input(f"Enter the remote path of file {i+1} from Server 1: "))
destination_directory_server1 = '/root/Desktop/Received'
hostname_server1 = '192.168.152.129'
username_server1 = 'crypticse'
password_server1 = '********'  # redacted
transfer_files(source_files_server1, destination_directory_server1,
               hostname_server1, username_server1, password_server1)

# ----- Server 2 -----
num_files_server2 = int(input("Enter the number of files to transfer from Server 2: "))
source_files_server2 = []
for i in range(num_files_server2):
    source_files_server2.append(input(f"Enter the remote path of file {i+1} from Server 2: "))
destination_directory_server2 = '/root/Desktop/Received'
hostname_server2 = '192.168.152.130'
username_server2 = 'crypticsec007'
password_server2 = '********'  # redacted
transfer_files(source_files_server2, destination_directory_server2,
               hostname_server2, username_server2, password_server2)
```

6.7 Decryption

The sixth stage decrypts the retrieved fragments. Selecting option 6 prompts for the key file (`encryption_key.xlsx`) and the fragment paths; each `.enc` fragment is decrypted

back to its original form (Figures 6.12 and 6.13).

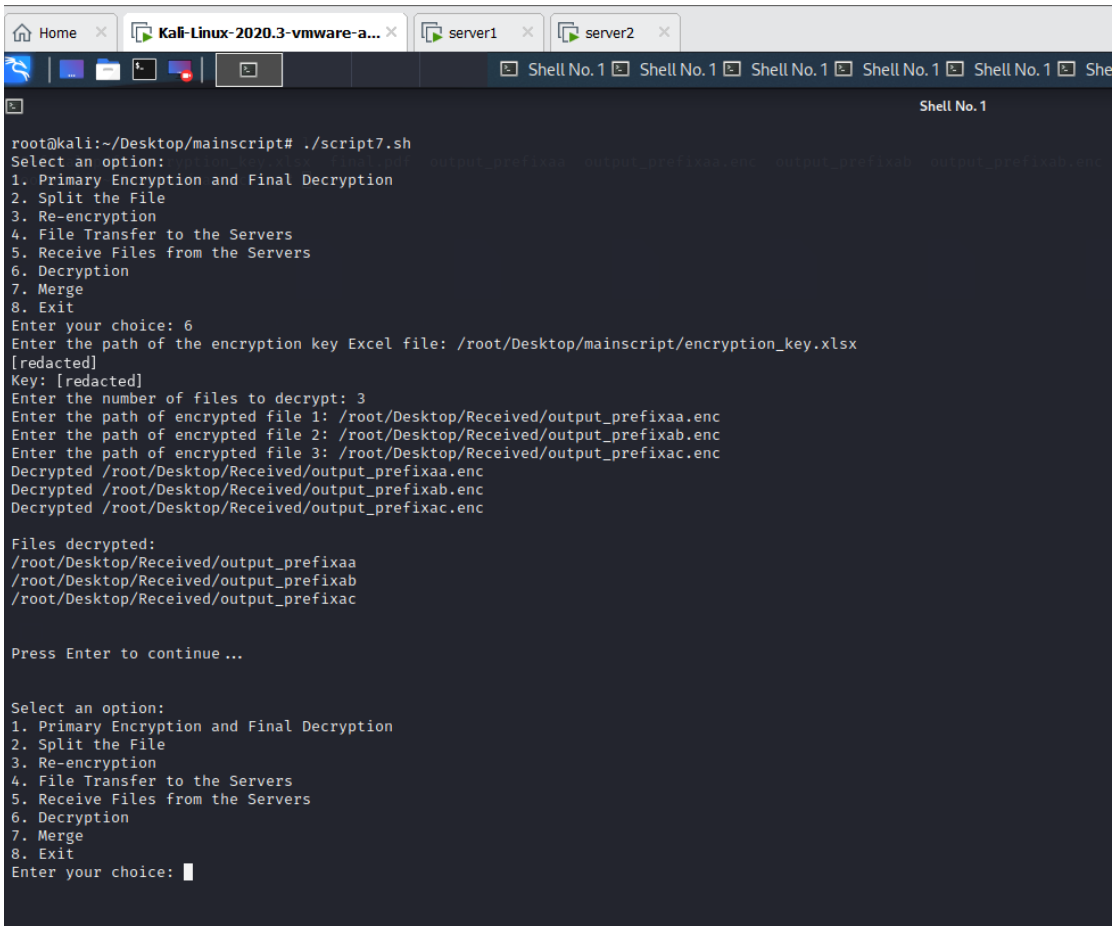


Figure 6.12: Decrypting the retrieved fragments (the key value is redacted).

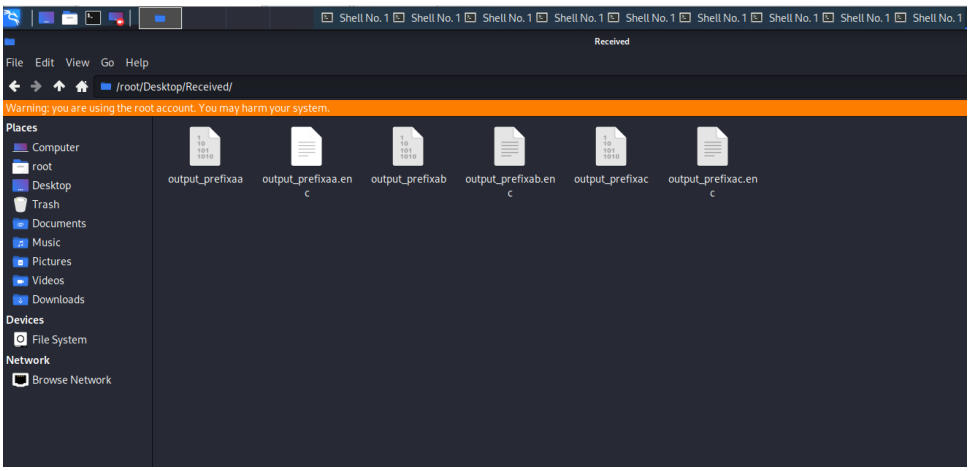


Figure 6.13: The decrypted fragments.

The decryption program, `dec1.py`, is shown in Listing 6.6.

Listing 6.6: Decryption of the re-encrypted fragments (`dec1.py`).

```python
import os
import base64
import openpyxl
from openpyxl import load_workbook
from cryptography.fernet import Fernet

def decrypt_file(filename, key):
  output_filename = os.path.splitext(filename)[0]
  with open(filename, 'rb') as file:
    data = file.read()
  cipher_suite = Fernet(key)
  decrypted_data = cipher_suite.decrypt(data)
  with open(output_filename, 'wb') as output_file:
    output_file.write(decrypted_data)
  return output_filename

# Load the encryption key from the Excel file
excel_filename = input("Enter the path of the encryption key Excel file: ")
workbook = load_workbook(excel_filename, read_only=True)
sheet = workbook.active
key = sheet['B1'].value

# Collect the encrypted fragment paths
filepaths = []
num_files = int(input("Enter the number of files to decrypt: "))
for i in range(num_files):
  filepaths.append(input(f"Enter the path of encrypted file {i+1}: "))

# Decrypt each fragment
decrypted_files = []
for filepath in filepaths:
  try:
    decrypted_files.append(decrypt_file(filepath, key))
    print(f"Decrypted {filepath}")
  except ValueError:
    print(f"Failed to decrypt {filepath}. Invalid key.")

print("\nFiles decrypted:")
for decrypted_file in decrypted_files:
  print(decrypted_file)
```

6.8 Merging the Fragments

The seventh stage reassembles the decrypted fragments into a single file. Selecting option 7 prompts for the output path and the fragment paths; entering `done` ends the loop and the fragments are concatenated into `Mergedfile` (Figures 6.14 and 6.15). This stage uses the Linux `cat` utility, integrated into the automation script (see `execute_merge` in Listing 6.1).

```
Select an option:
1. Primary Encryption and Final Decryption
2. Split the File
3. Re-encryption
4. File Transfer to the Servers
5. Receive Files from the Servers
6. Decryption
7. Merge
8. Exit
Enter your choice: 7
Enter the path and name of the merged file: /root/Desktop/Received/Margedfile
Enter the path of the split file to merge (or 'done' to finish): /root/Desktop/Received/output_prefixaa
Enter the path of the split file to merge (or 'done' to finish): /root/Desktop/Received/output_prefixab
Enter the path of the split file to merge (or 'done' to finish): /root/Desktop/Received/output_prefixac
Enter the path of the split file to merge (or 'done' to finish): done
Files merged successfully.


Press Enter to continue ...


Select an option:
1. Primary Encryption and Final Decryption
2. Split the File
3. Re-encryption
4. File Transfer to the Servers
5. Receive Files from the Servers
6. Decryption
7. Merge
8. Exit
Enter your choice:
```

Figure 6.14: Merging the decrypted fragments.

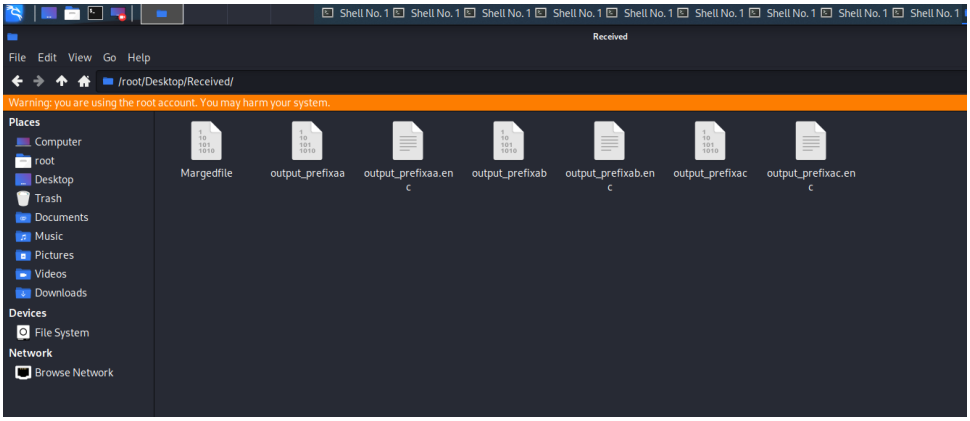


Figure 6.15: The merged file stored locally.

6.9 Final Decryption

The eighth and final stage recovers the original document. Selecting option 1 and then the decrypt action, with the same integer key used earlier (23) and the merged file, reverses the primary XOR encryption and restores the original `final.pdf` (Figures 6.16 and 6.17). This confirms that the file recovered at the end of the pipeline is byte-for-byte identical to the file that entered it.

```
Select an option:
1. Primary Encryption and Final Decryption
2. Split the File
3. Re-encryption
4. File Transfer to the Servers
5. Receive Files from the Servers
6. Decryption
7. Merge
8. Exit
Enter your choice: 1
Please select you option.
1. Encrypt File
2. Decrypt File
3. Quit
2
Enter a key as int!
23
Enter filename with extension:
/root/Desktop/Received/Margedfile
Please select you option.
1. Encrypt File
2. Decrypt File
3. Quit
```

Figure 6.16: Performing the final decryption.

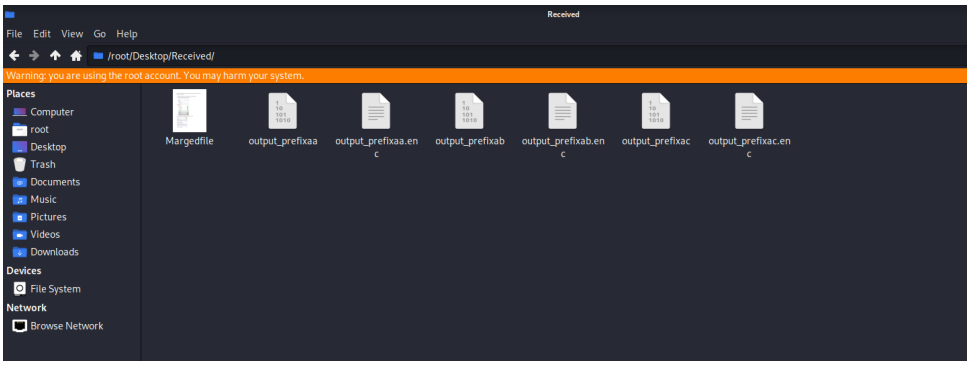


Figure 6.17: The recovered original file.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

This project presented a secure erasure code–based cloud storage system with secure data forwarding. By combining erasure coding with layered encryption and re-encryption, the system delivers reliable data security together with encrypted communication for cloud storage. Erasure coding and secure transmission together ensure the availability, durability, confidentiality, and integrity of data; the result is improved fault tolerance, successful information recovery, and protection against unauthorised access and data breaches. Because the data is encrypted, split, and re-encrypted before being distributed across independent servers, the system is substantially more secure than a conventional single-server cloud deployment, and the end-to-end demonstration confirmed that the original file can be recovered intact. Overall, the system provides a safe and dependable way to store and retrieve data in the cloud.

7.2 Future Work

Several directions could extend this work. Performance could be improved and formally benchmarked; data-integrity verification and auditing could be added; and the system could be generalised to multi-cloud environments with cost optimisation. User experience, accessibility, and collaborative sharing features could be developed, and a full data-lifecycle management capability — covering migration and secure disposal — could be introduced. Stronger key management (for example, hardware-backed keys or a threshold key-server deployment) and additional privacy measures would further harden the system. These directions aim to improve the functionality, security, effectiveness, and usability of secure cloud storage.

References

[1] P. Druschel and A. Rowstron, "PAST: A large-scale, persistent peer-to-peer storage utility," in *Proc. 8th Workshop on Hot Topics in Operating Systems (HotOS VIII)*, pp. 75–80, 2001.

[2] A. Haeberlen, A. Mislove, and P. Druschel, "Glacier: Highly durable, decentralized storage despite massive correlated failures," in *Proc. 2nd Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 143–158, 2005.

[3] Z. Wilcox-O'Hearn and B. Warner, "Tahoe: The least-authority filesystem," in *Proc. 4th ACM International Workshop on Storage Security and Survivability (StorageSS)*, pp. 21–26, 2008.

[4] A. Adya, W. J. Bolosky, M. Castro, G. Cermak, R. Chaiken, J. R. Douceur, J. Howell, J. R. Lorch, M. Theimer, and R. Wattenhofer, "Farsite: Federated, available, and reliable storage for an incompletely trusted environment," in *Proc. 5th Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 1–14, 2002.

[5] J. Kubiatowicz, D. Bindel, Y. Chen, P. Eaton, D. Geels, R. Gummadi, S. Rhea, H. Weatherspoon, W. Weimer, C. Wells, and B. Zhao, "OceanStore: An architecture for global-scale persistent storage," in *Proc. 9th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 190–201, 2000.

[6] S. C. Rhea, P. R. Eaton, D. Geels, H. Weatherspoon, B. Y. Zhao, and J. Kubiatowicz, "Pond: The OceanStore prototype," in *Proc. 2nd USENIX Conference on File and Storage Technologies (FAST)*, pp. 1–14, 2003.

[7] R. Bhagwan, K. Tati, Y.-C. Cheng, S. Savage, and G. M. Voelker, "Total recall: System support for automated availability management," in *Proc. 1st Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 337–350, 2004.

[8] A. G. Dimakis, V. Prabhakaran, and K. Ramchandran, "Decentralized erasure codes for distributed networked storage," *IEEE Transactions on Information Theory*, vol. 52, no. 6, pp. 2809–2816, 2006.

[9] A. G. Dimakis, V. Prabhakaran, and K. Ramchandran, "Ubiquitous access to distributed data in large-scale sensor networks through decentralized erasure codes," in *Proc. 4th International Symposium on Information Processing in Sensor Networks (IPSN)*, pp. 111–117, 2005.

[10] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang, and K. Fu, "Plutus: Scalable secure file sharing on untrusted storage," in *Proc. 2nd USENIX Conference on File and Storage Technologies (FAST)*, pp. 29–42, 2003.

[11] M. Mambo and E. Okamoto, "Proxy cryptosystems: Delegation of the power to decrypt ciphertexts," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E80-A, no. 1, pp. 54–63, 1997.

[12] M. Blaze, G. Bleumer, and M. Strauss, "Divertible protocols and atomic proxy cryptography," in *Proc. International Conference on the Theory and Application of Cryptographic Techniques (EUROCRYPT)*, pp. 127–144, 1998.

[13] H.-Y. Lin and W.-G. Tzeng, "A secure decentralized erasure code for distributed network storage," *IEEE Transactions on Parallel and Distributed Systems*, vol. 21, no. 11, pp. 1586–1594, 2010.

[14] D. R. Brownbridge, L. F. Marshall, and B. Randell, "The newcastle connection or UNIXes of the world unite!," *Software: Practice and Experience*, vol. 12, no. 12, pp. 1147–1162, 1982.